Available online at www.sciencedirect.com
SCIENCE DIRECT®
Computers & Operations Research 32 (2005) 1849–1865
computers & operations research
www.elsevier.com/locate/dsw

# Earliness–tardiness scheduling with setup considerations

Francis Sourd*

*Université Pierre et Marie Curie-LIP6 4, place Jussieu, 75252 Paris Cedex 05, France*

**Abstract**

The one-machine scheduling problem with sequence-dependent setup times and costs and earliness–tardiness penalties is addressed. A time-indexed formulation of the problem is presented as well as different relaxations that give lower bounds of the problem. Then, a branch-and-bound procedure based on one of these lower bounds is presented. The efficiency of this algorithm also relies on new dominance rules and on a heuristic to derive good feasible schedules. Computational tests are finally presented.


*Keywords:* Earliness–tardiness scheduling; Setup times; MIP formulation; Branch-and-bound; Heuristics

## 1. Introduction

The study of scheduling problems with earliness and tardiness penalties is motivated by the just-in-time (JIT) policy, which is based on the idea that early as well as late delivery must be discouraged. Much research efforts have been devoted to the study and the resolution of these problems for about 15 years. Another field of scheduling research, which is also motivated by practical considerations, is the minimization of setup costs and setup times in a schedule. Setups cannot be disregarded in many problems in which the production range is decomposed of a number of product groups such that almost no setup is required between the production of two products belonging to the same group, whereas long and expensive setup operations are required otherwise.

Whereas JIT policy tends to schedule the production according to the individual due dates of each order, setup considerations tend to group the production of orders belonging to the same family. Therefore, in most cases, earliness–tardiness cost minimization and setup optimization are conflicting and the minimization of the sum of these costs is highly difficult—the earliness–tardiness

* Fax: +33-1-44-27-70-00.
*E-mail address:* francis.sourd@lip6.fr (F. Sourd).

0305-0548/$ - see front matter © 2003 Elsevier Ltd. All rights reserved.
doi:10.1016/j.cor.2003.12.002

scheduling problem without setup and the setup minimization problem without earliness–tardiness costs are both NP-complete.

In this paper, we consider the following problem. A single machine has to process $n$ tasks $J_1, \ldots, J_n$ such that at most one task is performed at any time. Preemption of tasks is not allowed but idle time can be inserted between two tasks. Each task $J_i$ has a processing time $p_i$ and belongs to a group $g_i \in \{1, \ldots, q\}$ (with $q \leqslant n$). *Setup* or *changeover* times and costs, which are given as two $q \times q$ matrices, are associated to these groups. This means that in a schedule where $J_j$ is processed immediately after $J_i$, there must be a setup time of at least $s(g_i, g_j)$ time units between the completion time of $J_i$, denoted by $C_i$, and the start time of $J_j$, which is $C_j - p_j$. During this setup period, no other task can be performed by the machine and we assume that the cost of the setup operation is $c(g_i, g_j) \geqslant 0$. We assume that there is no setup time and no setup cost between tasks belonging to the same group (that is $s(g, g) = 0$ and $c(g, g) = 0$) and the setup matrices satisfy the triangle inequalities (that is $s(g, g') \leqslant s(g, g'') + s(g'', g')$ and $c(g, g') \leqslant c(g, g'') + c(g'', g')$). A machine is said to be *in state* $g$ as soon as the setup operation moving it into state $g$ is finished and it remains in state $g$ until another setup operation starts. Machine state is undefined during setup operations. Such setup times and costs are said to be *sequence-dependent* because they depend on both $g$ and $g'$. When $s(g, g')$ and $c(g, g')$ $(g \neq g')$ can be expressed as a function in only the state $g'$, setups are said to be *sequence-independent*.

In addition to the setup constraints and costs, earliness and tardiness costs are associated to each task. More precisely, $J_i$ has a due date $d_i$. If $C_i < d_i$, $J_i$ is said to be *early*, which is penalized by the cost $\alpha_i(d_i - C_i)$ (where $\alpha_i \geqslant 0$ is the earliness cost by unit time). Symmetrically, when $C_i > d_i$, $J_i$ is said to be *late*, which is penalized by the cost $\beta_i(C_i - d_i)$ (where $\beta_i \geqslant 0$ is the tardiness cost by unit time). The earliness–tardiness cost of $J_i$ will be denoted by $\mathrm{ET}_i = \max(\alpha_i(d_i - C_i), \beta_i(C_i - d_i))$. The problem is to minimize the sum of the earliness–tardiness and setup costs. In this paper, the problem will be referred to as earliness–tardiness scheduling with setups (ETSS).

For the simplicity of the presentation of the paper, we assume that all the tasks are available from time 0. However, all the properties presented in this paper can be immediately adapted in order to solve problems with distinct release date. For the simplicity of the formulation, we also assume—even if it is not realistic—that the machine has no predefined initial setup setting, that is no setup operation is required before the execution of the first task whatever the first task is. The results presented in this paper can trivially be adapted to deal with release dates and initial setup settings: we did it in our implementations.

This problem is NP-hard even when earliness and tardiness penalties are null or when there is neither setup times nor setup costs. Readers are referred to the works of Webster [1] and Chen [2] for a discussion about the complexity boundaries of these problems. Readers specially interested in earliness–tardiness scheduling are referred to the survey article by Baker and Scudder [3] and to the recent book by T'kindt and Billaut [4, chapter 5]. Those interested in setup considerations in scheduling problems are referred to the survey article by Allaverdi et al. [5].

Though the main issue in our problem is to find an optimal sequence, there is some interest in studying the subcases in which the sequence of tasks is assumed to be fixed. Indeed, when the sequence is fixed, setup operations are thereby fixed: their duration can be integrated into the processing times of the tasks and their total cost is a constant. So the problem is simply to find the completion times of the tasks to minimize the earliness–tardiness penalties. The problem, known as *timing problem*, can be formulated as a linear program [6] and this formulation is still valid while

all the deviation cost functions $ET_i$ are replaced by general piecewise linear convex cost functions. However, this problem can be more efficiently solved by a direct combinatorial algorithm based on a list scheduling approach [7–10]. More recently, it was shown that, by a dynamic programming approach, the problem is still polynomial when the cost functions are not assumed to be convex (but piecewise linear) [11]. The complexity of this algorithm depends on the number of segments of the cost functions in input. The timing problem is very important in practice because most scheduling algorithms first rank the tasks by the mean of either a (meta)heuristic or an enumeration scheme and then determine the optimal timing for the candidate sequence. For the single machine problem with earliness/tardiness penalties (without setups), we can refer to several branch-and-bound algorithms [12–14], heuristic algorithms [15,16], and algorithms to make feasible some unfeasible solutions of relaxed problems [14,17].

Some articles have been published about the subcase of ETSS where all the jobs have a common due date ($d_i = d$). Azizoglu and Webster [18] presented a branch-and-bound algorithm and a beam search procedure for the problem with sequence-independent setup times and an unrestricted common due date. More recently, Rabadi et al. [19] presented another branch-and-bound algorithm for the problem with sequence-dependent setup times whose lower bound is based on Kanet's algorithm. Genetic algorithms have also been proposed by Webster et al. [20].

For the generalization of ETSS problem where there are several parallel machines, 2 mixed-integer formulations have been proposed [21,22]. However, the reported experimentations show that they cannot be used to solve instances with more than 10 tasks.

ten Kate et al. [23] addressed the special case of our problem where $\alpha_i = \alpha$, $\beta_i = \beta$ and $c(g, g') = c$. They propose a branch-and-bound algorithm based on the algorithm of Yano and Kim [24]. In contrast, the branch-and-bound algorithm proposed in this paper is based on the lower bound of Sourd and Kedad-Sidhoum [14] which is better and more general than the one of Yano and Kim [24]. They also present some simple heuristic procedures.

The purpose of this work is to compare different algorithmic approaches to solve ETSS. The paper contains three main contributions. First, we propose a new, time-indexed, MIP formulation of the problem from which we derive lower bounds (one is based on a new valid cut, the second on a Lagrangean relaxation). Then, we present a branch-and-bound algorithm based on a weaker—but faster to compute—lower bound and on new dominance rules. Finally, we propose a new multi-start heuristic procedure.

In Section 2, we present a mixed-integer formulation of the problem with cuts that can be used to strengthen their continuous relaxations as well as a Lagrangean relaxation. In Section 3, we present a branch-and-bound algorithm with a new lower bound and new dominance rules. In Section 4, we present a simple but efficient heuristic algorithm as it is proved in Section 5 that presents experimental results for our algorithms.

## 2. MIP formulations

There are several possible ways of formulating ETSS as a MIP using "classical" formulations of scheduling problems. The reader is referred to the survey of Queyranne and Schulz [25] for a presentation of these MIP formulations of scheduling problems. Even if they are interested in regular criteria, some of these MIP can be directly adapted to cope with earliness–tardiness costs.

Table 1
Lower bounds based on MIP formulations

| Lower bound | Section | Value | CPU time (s) |
|---|---|---|---|
| Linear relaxation of the disjunctive model | 2.0 | $5 \times 10^{-6}$ | 0.2 |
| Linear relaxation of the TSP-based model | 2.0 | $2 \times 10^{-3}$ | 0.1 |
| Linear relaxation of the assignment model | 2.2 | 6314 | 200 |
| Above relaxation with cuts | 2.2 | 6454 | 2450 |
| Lagrangean relaxation $\max_\lambda \mathrm{LB}(\lambda)$ | 2.3 | 6301 | 15 |
| Lagrangean relaxation $\max_\lambda \mathrm{LB}_2(\lambda)$ | 2.3 | 6424 | 105 |
| Lower bound of the branch-and-bound | 3.2 | 5843 | 0.07 |
| Optimality proof (branch-and-bound) | 3 | 8902 | 3710 |

In a preliminary study of the problem, we formulated ETSS first using the disjunctive constraints (with "big $M$" values) and second using the so-called TSP variables. However, according to tests made with the MIP solver ILOG CPLEX [26], it is difficult to solve problems with more than 10 tasks. For example, solving an easy instance with 10 tasks and 2 groups, requires more than 15 min of computation. Moreover, the linear relaxations of both MIP are not good either: before cuts are added by ILOG CPLEX, the lower bound is in general quasi-null. Table 1 presents the values of the linear relaxations of both formulations for an instance with 20 jobs and 3 groups as well as the computation time. It also presents values for the other lower bounds that are presented later in the paper.

Since polynomial size formulation of ETSS are not satisfactory, we present in this section a time-indexed formulation of the problem. Indeed, linear relaxations of time-indexed formulations of scheduling problems often provide strong lower bounds [27,28]. Unfortunately, the sizes of these linear programs are large: they are pseudo-polynomial in the size of the input. These formulations require an upper bound on the makespan of an optimal schedule, denoted by $T$ and are called the *horizon* of the scheduling problem. In our problem, we can for example define it as $T = \max_i d_i + \sum_i p_i + \sum_i \max_j s(g_i, g_j)$. The last term of the expression is an upper bound on the total setup time for the problem. The tightest values may eventually be derived from the approach of Ventura and Radhakrishnan [17]. In the following, the pseudo-polynomial upper bounds are in fact polynomial in $T$, $n$ and $q$.

### 2.1. The assignment model

We are going to call this model the *assignment model* because it relies on the variables $x_{it}$ (for $1 \leqslant i \leqslant n$ and $0 \leqslant t < T$), which have the following meaning: $x_{it}$ is equal to 1 if task $J_i$ is executed between $t$ and $t+1$, and it is equal to 0 otherwise. For the problem without setup times, Sourd and Kedad-Sidhoum [14] have shown that the assignment costs $c_{it}$ can be defined so that the total assignment cost $\sum_{it} c_{it}x_{it}$ is a lower bound for the earliness–tardiness scheduling problem. An assignment is said to be *non-preemptive* when, for each job $J_i$, the set $\{t | x_{it} = 1\}$ is a single interval of $\mathbb{N}$ of length $p_i$. Sourd and Kedad-Sidhoum [14] have shown that the total assignment cost for a non-preemptive assignment is equal to the earliness–tardiness cost of the schedule derived from the assignment.

In order to take into account the setup types, the following elements of the earliness–tardiness problem MIP formulation are introduced:

- the boolean variables $y_{it}$ for $1 \leqslant i \leqslant q$ and $0 \leqslant t < T$. $y_{it}$ is equal to 1 if and only if the machine is idle during $[t, t+1)$ and is in state $i$;
- the boolean variables $s_{ijt}$ for $1 \leqslant i, j \leqslant q$ and $0 \leqslant t < T - s(i,j)$. $s_{ijt}$ is equal to 1 if and only if a setup operation that moves the machine from type $i$ to type $j$ starts at time $t$ and completes at $t + s(i,j)$;
- the boolean variables $z_{it}$ are eventually used to indicate that $J_i$ starts at $t$. In a preemptive assignment, $z_{it}$ is equal to 1 for each start time of a subpart of the execution of $J_i$ ($z_{it} = 1 \Leftrightarrow x_{it} = 1 \wedge x_{i,t-1} = 0$). Clearly, this variable is used to make the assignment non-preemptive.

In the following formulation, we assume that the boolean variables are null for the values of $t$ outside the definition intervals (e.g. $x_{i,-1} = 0$ and $s_{ijT} = 0$).

$$\min \quad \sum_{it} c_{it} x_{it} + \sum_{ijt} c(i,j) s_{ijt},$$

$$\text{s.t.} \quad \sum_{t} x_{it} = p_i, \quad \forall 1 \leqslant i \leqslant n, \tag{1}$$

$$\sum_{i=1}^{n} x_{it} + \sum_{i=1}^{q} y_{it} + \sum_{t' > t - s(i,j)}^{t' \leqslant t} s_{ijt'} = 1, \quad \forall 0 \leqslant t < T, \tag{2}$$

$$y_{i,t-1} + \sum_{g_j = i} x_{j,t-1} + \sum_{j \neq i} s_{j,i,t-s(j,i)} = y_{it} + \sum_{g_j = i} x_{jt} + \sum_{j \neq i} s_{ijt}, \quad \begin{matrix} \forall 0 < t < T, \\ \forall 1 \leqslant i \leqslant q, \end{matrix} \tag{3}$$

$$z_{it} \geqslant x_{it} - x_{i,t-1}, \quad \begin{matrix} \forall 1 < t < T, \\ \forall 1 \leqslant i \leqslant n, \end{matrix} \tag{4}$$

$$\sum_{t} z_{it} = 1, \quad \forall 1 \leqslant i \leqslant n, \tag{5}$$

$$x_{it},\ y_{it},\ z_{it},\ s_{ijt} \in \{0,1\}. \tag{6}$$

Eq. (1) ensures that each task is completely executed. Eq. (2) takes care of the resource constraints, that is at each time either exactly one task or one setup operation is executed or the machine is idle in exactly one state. Eq. (3) ensures that any change in the machine state corresponds to a setup operation. From Eq. (2), the left and right members of Eq. (3) are either equal to 1 or equal to 0; the left member is equal to 1 if and only if the machine is in state $i$ just before $t$ and the right member is equal to 1 if and only if the machine is in state $i$ at time $t$. Eq. (4) forces $z_{it}$ to be equal to 1 if and only if task $J_i$ is processed during $[t, t+1)$ but is not processed during $[t-1, t)$. Therefore, Eq. (5) ensures that the assignment is non-preemptive.

This integer program can only be solved to optimality for small values of $T$ and $n$. For instance, with ILOG CPLEX 8.1 [26], a problem with 20 tasks of duration 1, 3 setup groups and setup times equal to 5 can be solved in about 1 h of computation time with a 1 GHz PC.

### 2.2. Linear relaxation

Unfortunately, most practical problems cannot be formulated using very short processing times and setup times. For such problems, the linear relaxation of this integer program is a linear program that is solved by ILOG CPLEX 8.1 in no more than 200 s (see Table 1). The value of the linear relaxation can be improved by adding the following valid cut in the linear program, which is based on the observation that if $t \neq t'$ and $t \equiv t' \bmod p_i$, that is $t = t' + k p_i$ with $k \in \mathbb{Z} - \{0\}$, then in a non-preemptive assignment, $x_{it} = x_{it'} = 1$ is impossible. Otherwise, the processing time of $J_i$ would be strictly greater than $|t - t'| \geqslant p_i$.

Conversely, for any $t'$, for any non-preemptive schedule, there clearly exists some $t$ such that $t \equiv t' \bmod p_i$ and $x_{it} = 1$. Therefore, for any $0 \leqslant t' < p_i$, we have the valid cut

$$\sum_{t=t' \bmod p_i} x_{it} = 1. \tag{7}$$

Note that this valid cut is related to the so-called non-preemptive cuts independently proposed by Baptiste and Demassey [29] for a RCPSP MIP formulation. It can also be observed that this valid cut can be used to improve the lower bound of Sourd and Kedad-Sidhoum [14] for the problem without setups. The computation of the lower bound can then be made by solving an assignment problem between $\sum_i p_i$ operations and $T$ time slots instead of solving a transportation problem between $n$ tasks and $T$ time slots. So the computation time of the lower bound increases from $O(n^2 T)$ to $O(T^3)$.

In our preliminary experimental tests, the introduction of these cuts in our model significantly improves the value of the lower bound: the duality gap is decreased by a few percent but the computation time is typically increased by a factor 10 as it is illustrated by the example in Table 1.

### 2.3. Lagrangean relaxation

The proposed Lagrangean relaxation is based on the removal of the non-preemption constraints (4) and (5)—as for the lower bound of Sourd and Kedad-Sidhoum [14]—and on the Lagrangean relaxation of Eq. (1) to which the multipliers $\lambda_i$ $(1 \leqslant i \leqslant n)$ are associated. Then, the Lagrangean problem is

$$\min \sum_i \lambda_i p_i + \sum_{it} (c_{it} - \lambda_i) x_{it} + \sum_{ijt} c(i,j) s_{ijt}$$

subject to the constraints (2), (3) and (6). Note that the first sum in the objective is a constant. We will now show that the Lagrangean problem is polynomial in $n$, $q$ and $T$.

We first observe that if $x_{it} = 1$ is a feasible solution then $c_{it} - \lambda_i \leqslant c_{i't} - \lambda_{i'}$ for any $J_{i'}$ such that $g_{i'} = g_i$. Otherwise, if $c_{it} - \lambda_i > c_{i't} - \lambda_{i'}$ for some $i' \neq i$, a better solution can be obtained by the following modification $x_{it} = 0$ and $x_{i't} = 1$. Therefore, we can introduce the so-called *state-variable* $X_{gt} = y_{gt} + \sum_{g_i = g} x_{it}$ which equals one if and only if a task in state $g$ is processed at time $t$ (and 0 otherwise). Following the above remark, the cost for scheduling such a task in state $g$ at this time is $C_{gt} = \min(0, \min_{g_i = g} c_{it} - \lambda_i)$ and the Lagrangean problem becomes, after removal of the constant

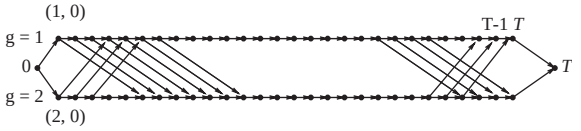


Fig. 1. Graph $G$ for a problem with $q = 2$.

term in the objective function,

$$\min \quad \sum_{gt} C_{gt}X_{gt} + \sum_{gg't} c(g,g')s_{gg't}, \tag{8}$$

$$\text{s.t.} \quad \sum_{g=1}^{q} X_{gt} + \sum_{t' > t-s(g,g')}^{t' \leqslant t} s_{gg't'} = 1, \quad \forall 0 \leqslant t < T, \tag{9}$$

$$X_{g,t-1} + \sum_{g' \neq g} s_{g',g,t-s(g',g)} = X_{gt} + \sum_{g' \neq g} s_{gg't}, \quad \begin{matrix} \forall 0 < t < T, \\ \forall 1 \leqslant g \leqslant q, \end{matrix} \tag{10}$$

$$X_{jt}, s_{ijt} \in \{0,1\}. \tag{11}$$

We show that any feasible solution of this linear programming can be regarded as a path in a graph $G(V \cup \{0,T\}, A_p \cup A_s \cup A_d)$ where $V = \{(g,t) | 1 \leqslant g \leqslant q, 0 \leqslant t \leqslant T\}$ and the set of arcs is partitioned into the *processing arcs* $A_p = \{((g,t),(g,t+1)) | 1 \leqslant g \leqslant q, 0 \leqslant t < T\}$ and the *setup arcs* $A_s = \{((g,t),(g',t+s(g,g'))) | 1 \leqslant g,g' \leqslant q, g \neq g', 0 \leqslant t \leqslant T - s(g,g')\}$ and the *dummy arcs* $A_d = \{(0,(g,0)),((g,T),T) | 1 \leqslant g \leqslant q\}$. Any processing arc $((g,t),(g,t+1))$ is associated to the cost $c_{gt}$ and the variable $X_{gt}$, any setup arc $((g,t),(g',t+s(g,g')))$ is associated to the cost $c(g,g')$ and to the variable $s_{gg't}$ and each dummy arc has no cost (and is related to no variable of the MIP). Note that when an initial setup state $g_0$ is specified, we simply have to remove from $A_d$ the arcs $(0,(g,0))$ for $g \neq g_0$. Fig. 1 depicts such a graph $G$ for a problem instance with $q = 2$, $s(1,2) = 5$ and $s(2,1) = 3$. By extension, we are going to say that a node $(g,t) \in V$ is in state $g$.

Clearly, a path from 0 to $T$ in $G$ corresponds to a solution of the Lagrangean problem that satisfies the constraints (9)–(11). Conversely, any feasible solution of the Lagrangean problem is a flow in $(V, E_p \cup E_s)$ by (10), while (9) and (11) force this flow to be a single path which is immediately transformed into a path from 0 to $T$ in $G$. Therefore, the Lagrangean problem can be solved by computing the shortest path from 0 to $T$ in $G$. It can be done in $O(\max(n,q^2)T)$ time: the graph has $O(q^2T)$ arcs and computing all the costs $C_{gt} = \min(0, \min_{g_i=g} c_{it} - \lambda_i)$ requires $O(nT)$ time.

For any fixed value of $\lambda=(\lambda_1,\ldots,\lambda_n)$, the solution of the Lagrangean problem is a lower bound, denoted by $\mathrm{LB}(\lambda)$, for ETSS. It is well known that the function $\lambda \mapsto \mathrm{LB}(\lambda)$ is concave and non-smooth. So, in order to have a lower bound as good as possible, the function LB can be maximized with a subgradient algorithm.

When setup times or costs are large, it can be observed that the shortest path for the Lagrangean problem usually traverses vertices that belong to only one or two different groups. By contrast, a path corresponding to a feasible schedule should traverse *all* the states in $\{g_i | 1 \leqslant i \leqslant n\}$. Therefore, by analogy with [28], we can add to the Lagrangean problem the constraint that the path must traverse at least $k$ different states. Assuming that the value $k$ is fixed, such a shortest path can be computed in polynomial time yielding a lower bound $\mathrm{LB}_k(\lambda)$ but the algorithm is exponential in $k$.

However, according to our preliminary experimental tests, $LB_2(\lambda)$ is only slightly better than $LB(\lambda)$ but require significantly more computation time (see Table 1).

In practice, this Lagrangean relaxation renders a lower bound that is only slightly weaker than the linear relaxation of the time-indexed MIP but that can be computed significantly faster. However, with the aim of building a branch-and-bound algorithm in view, this lower bound is too time consuming. Therefore, in the next section, a weaker lower bound is presented and used in the branch-and-bound algorithm.

## 3. Branch-and-bound algorithm

In this section, we present an adaptation of the branch-and-bound algorithm presented by Sourd [30] to solve the one-machine earliness–tardiness problem without setups.

### 3.1. Overview of the algorithm

The branching scheme of the branch-and-bound algorithm consists in constructing the sequence from the first task in the sequence to the last one. Each node of the enumeration tree is associated with a partial initial sequence $\sigma = (\sigma_1, \ldots, \sigma_k)$, where the length of the sequence $k = |\sigma|$ is also the depth of the node in the enumeration tree. This node has $n - k$ child nodes, one for each job $J_i$ that is not in the partial sequence $\sigma$. The child node associated to $J_i$ corresponds to the sequence $\sigma \oplus i = (\sigma_1, \ldots, \sigma_k, i)$. At a leaf node, the initial sequence contains the $n$ tasks of the problem and the optimal schedule corresponding to the sequence can polynomially be derived with the optimal timing algorithm.

For any partial sequence $\sigma$, let $\Sigma(\sigma, t)$ be the minimum cost for scheduling the chain of tasks $J_{\sigma_1} \to \cdots \to J_{\sigma_k}$ such that the last task $J_{\sigma_k}$ is completed before time $t$. It has been shown that the function $t \mapsto \Sigma(\sigma, t)$ is piecewise linear with $O(n)$ segments and it is convex and nonincreasing [11]. Furthermore, when the function $\Sigma(\sigma, t)$ is known, the function $\Sigma(\sigma \oplus i, t)$ can be computed in $O(n)$ time [11] so it is efficient to dynamically compute this during the search in the enumeration tree. At a leaf node, the minimal cost of the schedule is simply given by $\lim_{t\to\infty} \Sigma(\sigma, t)$, which will be denoted by $\Sigma(\sigma)$. The function will also be very useful to detect dominated partial schedules in Section 3.3.

### 3.2. Lower bound

Here we describe the computation of the lower bound at a node of the branch-and-bound algorithm. The partial initial sequence is still denoted by $\sigma$ and, $k$ denotes the length of $\sigma$. The last task in this initial sequence belongs to the group $g_{\sigma_k}$, more simply denoted by $g_\sigma$. Let us denote by $J(\sigma) = \{J_{\sigma(1)}, \ldots, J_{\sigma(k)}\}$ the set of tasks in $\sigma$ and, for simpler notations, let us assume without loss of generality that the set of the $n - k$ non-sequenced tasks is $J - J(\sigma) = \{J_1, \ldots, J_{n-k}\}$.

Let $\bar{p}_0$ be a lower bound of the total setup time required to compute the tasks $J_1, \ldots, J_{n-k}$ after the initial sequence $\sigma$. Classically, $\bar{p}_0$ can be given by the length of the shortest Hamiltonian path (starting at $g_\sigma$) in a digraph representing the possible setup operations: the nodes of the digraph are the groups $\{1, \ldots, q\}$, the set of arcs is $\{(g_i, g_j) | 1 \leqslant i \leqslant n-k, 1 \leqslant j \leqslant n-k\}$. Computing such a $\bar{p}_0$

is NP-complete but $q$ is usually very small so that the shortest Hamiltonian path can be computed by a simple enumerative algorithm. If $q$ is too large, say greater than 5 or 6, a weaker value can be taken for $\bar{p}_0$ as the minimum spanning tree in this graph, which can be computed in polynomial time.

In the computation of the lower bound, setup periods and idleness periods will be considered equally: whenever a machine does not process a task, we will say the machine is *inactive* whatever the reason—setup or deliberate idleness—is. We now identify a time interval in which a total inactivity of length at least $\bar{p}_0$ is required in any feasible schedule. Let $p^g = \sum_{1 \leqslant i \leqslant n-k}^{g_i=g} p_i$ be the total processing time of the unsequenced tasks in group $g$. Clearly, in a feasible schedule, in any time interval of length $\ell = \bar{p}_0 + \sum_{i=1}^{n-k} p_i - \min_g p^g$, there is at least a total inactivity of length $\bar{p}_0$, otherwise the setup constraints would be violated. So, by introducing $s = p_\sigma = \sum_{J_i \in J(\sigma)} p_i$ and $e = s + \ell$, we have that any feasible schedule satisfies the so-called "idle time insertion" constraint that says the total idle time between $s$ and $e$ is at least $\bar{p}_0$.

By inserting this new constraint in the mathematical program formulated by Sourd [11], the following program renders a lower bound for ETSS in the presence of an initial partial sequence $\sigma$. In this program, the variables are the functions $\delta_i$: $[p_\sigma, T] \to 0, 1$. For $i \geqslant 1$, $\delta_i(t) = 1$ if $J_i$ is being processed at $t$ and $\delta_i(t) = 0$ otherwise. $\delta_0$ indicates whether the machine is inactive or not. These function variables are piecewise constant and we assume that they have a finite number of breakpoints so that there is no problem about the existence of the integrals.

$$\inf \quad \int_{p_\sigma}^{T} \sum_{i=0}^{n-k} \delta_i(t) f_i(t)\, \mathrm{d}t, \tag{12}$$

$$\text{s.t.} \quad \int_{p_\sigma}^{T} f_i(t)\, \mathrm{d}t = p_i, \quad \forall i \in \{1, \ldots, n-k\}, \tag{13}$$

$$\sum_{i=0}^{n-k} \delta_i(t)\, \mathrm{d}t = 1, \quad \forall t \in [p_\sigma, T], \tag{14}$$

$$\int_{s}^{e} \delta_0(t)\, \mathrm{d}t \geqslant \bar{p}_0. \tag{15}$$

In the formulation of the objective function (12), the function $f_0$ represents the idleness costs and is given (for $t > p_\sigma$) by $f_0(t) = \partial\Sigma(\sigma, t)/\partial t$ and, for any task $J_i$ (with $i > 0$), $f_i$ is a piecewise linear cost function which is derived from the parameters associated to $J_i$. In practice, there are several possible ways to build the functions $f_i$ so that the above program is a lower bound but, in order to fix the ideas, we are going to define $f_i$ as $f_i(t) = \alpha_i((d_i - t)/p_i - \frac{1}{2})$ for $p_\sigma \leqslant t \leqslant d_i$ and $f_i(t) = \beta_i((t - d_i)/p_i + \frac{1}{2})$ for $d_i < t \leqslant T$. Eq. (13) ensure that each task is completely executed while Eq. (14) formulate the capacity constraints. Eq. (15) is the "idle time insertion" constraint.

In the absence of setup, the computation of the lower bound was based on a strong duality result [30] but, due to the "idle time insertion" constraint, we adopt here a Lagrangean relaxation. By making a Lagrangean relaxation of the duration constraints (13), we have that, for any

$(u_1,\ldots,u_{n-k}) \in \mathbb{R}^{n-k}$, the solution of the following program is still a lower bound of the total cost:

$$\inf \quad \sum_i u_i p_i + \int_{p_\sigma}^{T} \sum_{i=1}^{n-k} \delta_i(t)(f_i(t) - u_i)\,\mathrm{d}t + \int_{p_\sigma}^{T} \delta_0(t) f_0(t)\,\mathrm{d}t,$$

$$\text{s.t.} \quad (14) \text{ and } (15).$$

After introducing the value $u_0 = 0$, we have the simpler formulation

$$\inf \quad \sum_i u_i p_i + \int_{p_\sigma}^{T} \sum_{i=0}^{n-k} \delta_i(t)(f_i(t) - u_i)\,\mathrm{d}t, \tag{16}$$

$$\text{s.t.} \quad (14) \text{ and } (15).$$

For any $(u_1,\ldots,u_{n-k})$, let $\mathscr{L}(u_1,\ldots,u_{n-k})$ denote the optimal value of the above program. We are going to show, that when the Lagrangean variables are fixed, $\mathscr{L}(u_1,\ldots,u_{n-k})$ can be computed in polynomial time. We have to build a "pseudo-schedule" that may violate the duration constraints but must satisfy the "idle time insertion" constraint and minimize the cost criterion given in (16). The following lemma gives a partial characterization of an optimal pseudo-schedule.

**Lemma 1.** *There exists a dominant optimal pseudo-schedule such that*

- *if* $f_0(t) \leqslant \min_{1\leqslant i\leqslant n-k} f_i(t) - u_i$ *then* $\delta_0(t) = 1$,
- *if* $\delta_j(t) = 1$, *then we have* $f_j(t) - u_j \leqslant \min_{i\geqslant 1} f_i(t) - u_i$,
- *if* $\delta_j(t) = 1$ *for* $t \notin [s,e]$, *then we have* $f_j(t) - u_j \leqslant \min_{i\geqslant 0} f_i(t) - u_i$.

**Proof.** If the pseudo-task $J_i$ is scheduled on some time interval $I$ such that $f_0(t) \leqslant f_i(t) - u_i$ for any $t \in I$. We can modify the pseudo-schedule so that the machine is idle on $I$ without violating the "idle time insertion" constraint and without increasing the total cost. This proves that there exists an optimal pseudo-schedule such that $\delta_0(t) = 1$ whenever $f_0(t) \leqslant f_i(t) - u_i$. The two other assertions are similarly proved. □

Conversely, in an optimal pseudo-schedule, there may be time intervals $I$ and pseudo-tasks $J_i$ such that $f_0(t) > f_i(t) - u_i$ for any $t \in I$ but $\delta_0(t) = 1$. That is idle time is inserted even if it would have been less costly to process $J_i$ instead. This is due to the "idle time insertion" constraint of the pseudo-schedule.

We now rewrite the cost criterion in order to transform the problem into the minimization of the cost of the idle time insertion. Let $v(t) = \min_{0\leqslant i\leqslant n-k} f_i(t) - u_i$. Because of the equality constraint (14), we can rewrite the objective function as

$$\int_{p_\sigma}^{T} \sum_{i=0}^{n} \delta_i(t)(f_i(t) - u_i - v(t))\,\mathrm{d}t + \int_{p_\sigma}^{T} v(t)\,\mathrm{d}t. \tag{17}$$

When $(u_1,\ldots,u_n)$ is fixed, the second integral of (17) is a constant. Moreover, Lemma 1 says that if $\delta_i(t) = 1$ (for some $i > 0$) then $f_i(t) - u_i = v(t)$ and if $\delta_0(t) = 1$ with $t \notin (s,e)$ then $f_0(t) - u_0 = v(t)$. So, by removing the constant terms in expression (17), we have that an optimal pseudo-schedule minimizes $\int_s^e \delta_0(t) h_0(t)\,\mathrm{d}t$ (with $h_0(t) = f_0(t) - v(t) \geqslant 0$) subject to the constraint $\int_s^e \delta_0(t)\,\mathrm{d}t \geqslant \bar{p}_0$.

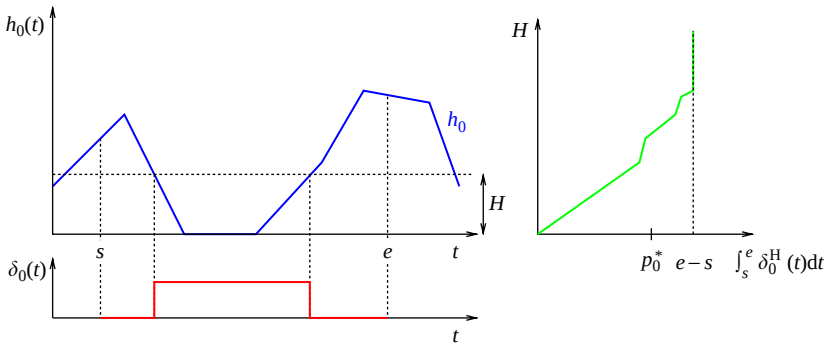


Fig. 2. Computing $\delta_0(t)$ between $s$ and $e$.

**Lemma 2.** *A solution of minimizing* $\int_s^e \delta_0(t)h_0(t)\,dt$ *subject to the constraint* $\int_s^e \delta_0(t)\,dt \geqslant \bar{p}_0$ *is of the form*

$$\delta_0^H(t) = \begin{cases} 1 & \text{if } h_0(t) \leqslant H, \\ 0 & \text{otherwise}, \end{cases}$$

*for some value* $H$.

**Proof.** We first observe that for any value $H$, $\delta_0^H$ has a finite number of segments. The lemma is shown by considering a function $\delta_0$ such that there exists $H$, $\varepsilon > 0$ and $t_1, t_2 \in [s, e-\varepsilon]$ such that

- $h_0(t) > H$ and $\delta_0(t) = 1$ for all $t \in (t_1, t_1+\varepsilon)$ and
- $h_0(t) \leqslant H$ and $\delta_0(t) = 0$ for all $t \in (t_2, t_2+\varepsilon)$.

We observe that a better feasible solution is achieved by setting "$\delta_0(t) \leftarrow 1 - \delta_0(t)$" for any $t \in (t1, t1+\varepsilon) \cup (t2, t2+\varepsilon)$. □

When $H$ increases from 0 to $\infty$, the function $\varphi : H \mapsto \int_s^e \delta_0^H(t)\,dt$ continuously increases from 0 to $e-s$ (see Fig. 2), which shows that for some $H^\star$, $\varphi(H^\star) = \bar{p}_0$. Computationally speaking, the function $\varphi$ is locally linear in the neighborhood of $H$ when there is no breakpoint $(t, h_0(t))$ of $h_0$ such that $h_0(t) = H$. Therefore, the number of breakpoints of $\varphi$ is bounded by the number of breakpoints of $h_0$ in the interval $[s, e]$ (note that $s$ and $e$ are considered as breakpoints), which is in $O(n)$. So, assuming $h_0$ is built, the value $H^\star$ can be computed in $O(n)$ and the function $\delta_0$ on the interval $[s, e]$ is then build in $O(n)$. Finally, the relaxed problem (16) subject to (14) and (15) is solved in $O(n^2)$, which represents the complexity for computing the function $v$.

We have shown that for any vector $(u_1, \ldots, u_{n-k})$ of Lagrangean multipliers, we have a lower bound $\mathscr{L}(u_1, \ldots, u_{n-k})$ for ETSS. Classically, this function is concave and can be maximized by several classical methods (see e.g. [31]) in order to obtain the best possible lower bound. In our implementation, we used a simple subgradient method. A good starting point for this method can be derived from the Lagrangean vector computed at the parent node in the branch-and-bound tree. A good trade-off between the quality of the lower bound and the computation time is obtained by limiting the search to about 10 iterations. The reader is referred to [11] for more details.

### 3.3. Dominance rules

In the problem without setup times, the gap between the lower bound at the root node of the branch-and-bound tree and the optimum is usually weak (around 5%) [30]. In the presence of setup types, despite the adaptation presented in Section 3.2, the gap is significantly greater (for the example in Table 1 the gap equals 34%). In order to compensate for this weaker bound, we improved the dominance rules used to solve the problem without setup times [14].

Let us consider a node of the enumeration tree to which the initial sequence $\sigma$ of length $k$ is associated. Let us denote by $\bar{J}(\sigma)$ the subset of $J(\sigma)$ restricted to the sequences $\sigma' \in J(\sigma)$ such that $g_\sigma = g_{\sigma'}$. Let us consider an initial sequence $\sigma' \neq \sigma$ in $\bar{J}(\sigma)$. In other words, $\sigma$ and $\sigma'$ share the same tasks and finishes with the same type. Let us assume that for any $t \geqslant p_\sigma$, $\Sigma(\sigma', t) \leqslant \Sigma(\sigma, t)$. Let us consider a feasible solution $C_1, \ldots, C_n$ starting by the initial sequence $\sigma$ and let $C_\sigma$ denote the completion time $C_{\sigma(k)}$ where $k = |\sigma|$. This schedule can then be modified by resequencing the first $k$ tasks in the order given by $\sigma'$ and by scheduling these $k$ tasks optimally subject to the constraint that these $k$ tasks are scheduled before $C_\sigma$. Since $g_\sigma = g_{\sigma'}$, the transition time and cost between $J_{\sigma'(k)}$ and $J_{\sigma(k+1)}$ is unchanged so that the execution times of the last $n-k-1$ tasks are not modified. Clearly, after the resequencing, the cost of the schedule is changed by $\Sigma(\sigma', C_\sigma) - \Sigma(\sigma, C_\sigma) \leqslant 0$.

Therefore, the initial sequence $\sigma$ is dominated by $\sigma'$ and, unless we have $\Sigma(\sigma', t) = \Sigma(\sigma, t)$ for all $t$, it is strictly dominated and the node associated to $\sigma$ can be discarded. When $\Sigma(\sigma', t) = \Sigma(\sigma, t)$ (for all $t$), one of the two nodes can be discarded, for example the second one in the lexicographic order.

This dominance rule can be improved as follows. Let us consider a set $S \subset \bar{J}(\sigma) - \{\sigma\}$ and let us assume that, for any $t$, $\Sigma(\sigma, t) \geqslant \Sigma_{\min}(S, t)$ where $\Sigma_{\min}(S, t)$ is a shorter notation for $\min_{\sigma' \in S} \Sigma(\sigma', t)$. Let us again consider a feasible solution $C_1, \ldots, C_n$ starting by the initial sequence $\sigma$. From the preceding assumption, there exists a partial sequence $\sigma'$ in $S$ such that $\Sigma(\sigma', C_\sigma) - \Sigma(\sigma, C_\sigma) \leqslant 0$. Consequently, the same transformation can be done, improving the cost of the schedule. So, a partial sequence can be discarded as soon as there exists a set $S \subseteq \bar{J}(\sigma) - \{\sigma\}$ such that, for any $t$, $\Sigma(\sigma, t) \geqslant \Sigma_{\min}(S, t)$ and, for some $t$, $\Sigma(\sigma, t) \neq \Sigma_{\min}(S, t)$.

In theory, $S$ should be equal to $\bar{J}(\sigma) - \{\sigma\}$ in order to have the better possible dominance rule. However, the size of such a set is at least $(k-1)!$ so, in a practical implementation, only a subset of polynomial size of $S(\sigma)$ can be considered. Heuristically, it seems interesting to choose for $S$ a neighborhood of $\sigma$. More formally, let us introduce the two following transformations—sometimes referred to as general pairwise interchange in the literature. The first one swap two tasks in the sequence and the second one removes a task from the sequence and reinsert it at another rank.

$$T^1_{ij}(\sigma) = (\sigma(1), \ldots, \sigma(i-1), \sigma(j), \sigma(i+1), \ldots, \sigma(j-1), \sigma(i), \sigma(j+1), \ldots, \sigma(k)),$$

$$T^2_{ij}(\sigma) = (\sigma(1), \ldots, \sigma(i-1), \sigma(i+1), \ldots, \sigma(j), \sigma(i), \sigma(j+1), \ldots, \sigma(k)).$$

For a given partial sequence $\sigma$, $S$ is then defined as the intersection set, denoted by $S^+(\sigma)$, between $S(\sigma)$ and $\{T^1_{ij}(\sigma) | i < j\} \cup \{T^2_{ij}(\sigma) | i \neq j\}$. The size of $S^+(\sigma)$ is in $O(k^2)$ so that the function $t \mapsto \Sigma(S^+(\sigma), t)$ has at most $O(k^3)$ segments [30]. Computing this function from scratch is $O(k^4)$ time but we now show that this computation time can be decreased to $O(k^3)$ by using the information computed at the parent node.

Let us consider the child node of $\sigma$ associated to the initial sequence $\sigma \oplus l$, whose length is therefore $k+1$. The set $S^+(\sigma \oplus l)$ can be partitioned into 2 subsets: $F = \{\sigma' \oplus l | \sigma' \in S^+(\sigma)\}$ and

its complementary $\bar{F}$. We clearly have $\Sigma_{\min}(S^+(\sigma \oplus l), t) = \min(\Sigma_{\min}(F,t), \Sigma_{\min}(\bar{F},t))$ and we are going to show that both functions in the min expression can be computed in $O(k^3)$ and have $O(k^3)$ segments. This result will show that the function $t \mapsto \Sigma_{\min}(S^+(\sigma \oplus l), t)$ is then computed in $O(k^3)$. Let us first consider the left function in the min expression. Based on [30], we establish the following relation between the partial schedules in $F$ and in $S^+(\sigma)$:

$$\Sigma_{\min}(F,t) = \min_{t' \leqslant t - p_i - s(g_\sigma, g_l)} (\Sigma_{\min}(S^+(\sigma), t')) + \mathrm{ET}_l(t)$$

$$= \Sigma_{\min}(S^+(\sigma), t - p_i - s(g_\sigma, g_l)) + \mathrm{ET}_l(t).$$

The second equality comes from the fact that the function $t \mapsto \Sigma(S^+(\sigma), t)$ is non-increasing. As $t \mapsto \Sigma(S^+(\sigma), t)$ has $O(k^3)$ segments, the function $t \mapsto \Sigma_{\min}(F,t)$ can be computed in $O(k^3)$. Eventually, we observe that the size of $\bar{F}$ is in $O(k)$ since any sequence $T^1_{ij}(\sigma \oplus l)$ or $T^2_{ij}(\sigma \oplus l)$ in this set implies that either $i$ or $j$ is in $\{k, k+1\}$. Therefore, the function $t \mapsto \Sigma(\bar{F}, t)$ can be computed in $O(k^3)$ time, which eventually proves that $t \mapsto \Sigma(S^+(\sigma \oplus l), t)$ is computed with the same complexity.

However, in practice, we observe that the function $t \mapsto \Sigma(S^+(\sigma), t)$ has less than $k^3$ segments in average, which means that practical running times are faster than this worst case analysis.

### 3.4. Remark about the implementation

In our software implementation, the branch-and-bound algorithm has been adapted to solve problems with release dates. The state of the machine at time 0 can also be enforced, this constraint is often formulated in real benchmark problems provided by ILOG.

We also observe that our branch-and-bound algorithm does not use the property that the earliness–tardiness cost functions $\max(\alpha_i(d_i - C_i), \beta_i(C_i - d_i))$ are convex in $C_i$. Therefore, the algorithm can be adapted in order to solve problems with break constraints—see [32] for a formulation of break constraints and [11] for modeling these constraints with piecewise linear cost functions.

## 4. Feasible solutions

The performance of the branch-and-bound algorithms notoriously depends on the existence and the quality of a feasible solution: the better upper bound we have, the faster the algorithm will run. So, in this section we present a heuristic to compute such a solution.

Although good heuristics can be based on the lower bound of Sourd and Kedad-Sidhoum [14] to build good solutions for the problem without setup, unfortunately it is unlikely to build efficient heuristic based on the lower bounds introduced in the previous sections for problems with non-negligible setup times and costs. Indeed, for the linear relaxation of Section 2.2, the optimal solution is far from being integer and cannot be easily transformed into an integer solution: typically, in a problem with $q$ states, $y_g + \sum_{g_i = g} x_{git} \approx 1/q$ for any $g$ and any $t$, that is all the states are executed simultaneously as "in parallel" (without setup) instead of being serially executed one after the other (inducing setups). In the Lagrangean relaxation, some tasks are not "processed" in the optimal solution, even worse all the tasks belonging to some state may be missing in the solution: so inserting these jobs to build a feasible solution may be very tricky. Finally, in the lower bound used in the branch-and-bound algorithm presented in Section 3.2, solutions contain jobs in different

states that are scheduled without setup operations in between: inserting setup operations without re-grouping tasks that belong to the same group usually leads to very bad solutions. Moreover, we can observe that lower bound computation is quite time consuming since it is as long as thousands of runs of the optimal timing algorithm.

Therefore, we propose a local search-based algorithm to compute a good initial solution. Our heuristic is a simple iterative improvement procedure based on the union of the two neighborhoods introduced in Section 3.3 to discard the dominated partial sequences. As it is well known that there are no good priority rules for earliness–tardiness scheduling—at least for randomly generated instances—we simply initialized the improvement procedure with a random sequence. As the quality of the initial upper bound UB is important, this procedure is run several times (arbitrarily $n$ times) with different initial random sequences and the best solution is kept at the end. The details are given in Algorithm 1.

**Algorithm 1** Compute an initial solution

**for** $i = 1$ to $n$ **do**
  Initialize $\sigma'$ as a random permutation over $\{1,\ldots,n\}$
  $\mathrm{UB}_i = \Sigma(\sigma')$
  **repeat**
    $\sigma \leftarrow \sigma'$
    **for all** $i,j$ such that $i \neq j$ and $i,j \in \{1,\ldots,n\}$ **do**
      **if** $\Sigma(T^1_{ij}(\sigma)) < \mathrm{UB}_i$ and $i < j$ **then**
        $\sigma' \leftarrow T^1_{ij}(\sigma); \mathrm{UB}_i = \Sigma(T^1_{ij}(\sigma))$
      **end if**
      **if** $\Sigma(T^2_{ij}(\sigma)) < \mathrm{UB}_i$ **then**
        $\sigma' \leftarrow T^2_{ij}(\sigma); \mathrm{UB}_i = \Sigma(T^2_{ij}(\sigma))$
      **end if**
    **end for**
  **until** $\sigma = \sigma'$
**end for**
$\mathrm{UB} = \min_i \mathrm{UB}_i$

Although the strategy adopted in this algorithm if very basic, it was the most efficient one compared to simple implementation of other metaheuristics (tabu search and genetic algorithm) we also tested. It could be explained by the fact that there are a few local minima with respect to the neighborhood we defined and these minima are quite distant.

## 5. Experimental tests

In this section, we will present some computational results for the branch-and-bound algorithm developed in Section 3 and the initial heuristic introduced in Section 4. The algorithms were implemented in Microsoft Visual C++ 7. We tested our implementation on instances with $n = 10$, 15 and 20 jobs and $q = 2$, 3 and 4 different machine states. For each task, the processing time

Table 2
Average computation times

| | | | | | |
|---|---|---|---|---|---|
| Number of activities | $n=10$ | $n=12$ | $n=15$ | | |
| Mean CPU time | 0.96s | 7.3s | 159s | | |
| Number of batches | $q=2$ | $q=3$ | $q=4$ | | |
| Mean CPU time | 41.2s | 57.2s | 68.9s | | |
| Setup Length | Short | Long | | | |
| Mean CPU time | 9.4s | 102s | | | |
| Range factor | $\rho=0.2$ | $\rho=0.4$ | $\rho=0.6$ | $\rho=0.8$ | |
| Mean CPU time | 66.0s | 56.0s | 50.6s | 50.2s | |
| Tardiness factor | $\tau=0$ | $\tau=0.2$ | $\tau=0.4$ | $\tau=0.6$ | $\tau=0.8$ |
| Mean CPU time | 137s | 82.5s | 37.6s | 14.9s | 6.8s |

was generated from the uniform distribution $[10,100]$ and the earliness and tardiness penalties were generated from [5,29]. As usually done in the literature, the generation of due dates is described by two parameters, the *average tardiness* factor $\tau$ and the *range* factor $\rho$: due dates are then generated from the uniform distribution $[\max(0,P(1-\tau-\rho/2)),P(1-\tau+\rho/2)]$, with $P=\sum_i p_i$. Finally, we emphasized two classes of setup types for our study. In the first class (resp. second class), the setup costs and the setup times are generated from the uniform distribution $[10,20]$ (resp. $[100,200]$): these classes respectively corresponds to the short and long setup operations. We observe that the generation rule induces that triangle inequalities are satisfied for setup costs and times in both the classes. Finally, the group $g_i$ to which $J_i$ belongs, is chosen from the uniform distribution $\{1,\ldots,q\}$.

In this way, we generated 360 classes of instances parametrized by $n\in\{10;12;15\}$, $q\in\{2;3;4\}$, $\rho\in\{0.2;0.4;0.6;0.8\}$, $\tau\in\{0;0.2;0.4;0.6;0.8\}$ and a boolean indicating whether the setup operations are short or long. For each class, 6 instances were generated so that the experimental tests presented in this section are grounded on 2160 instances. All these instances have been solved by our algorithm with a mean computational time of 55 s (on a 2 GHz PC) with results ranging between 0.1 and 3085s. The heuristic described in Section 4 found the optimal solution in 95.2% of the instances with maximal computation times lower than 0.2 s. The average deviation between the solution found by the heuristic and the optimal solution is 0.08% and the maximal deviation is less than 10% for all the generated instances. Finally, for some of the instances for which the heuristic did not find the optimal solution, we ran again the heuristic with $10n$ (instead of $n$) executions of the descent procedure and the optimum was found in all these cases.

Table 2 presents how the difficulty of the instances varies according to the variation of the parameters. Each cell of the table shows the average computation time to solve the instances of a specified subclass. For example the first cell in the first line of the table says that the average time to solve the instances with $n=10$ tasks is 0.96 s. The table shows that the number of jobs $n$ is unsurprisingly the main indicator of the difficulty of the instances. Also, problems with long setup times are clearly more difficult than problems with short setups: this observation can be simply explained by the fact that the lower bound is the adaptation of a good lower bound for the problem without setups. The tardiness factor $\tau$ has also a great impact on the difficulty of the instances: when $\tau=0$, optimal schedules tend to have more early tasks and problems are then harder that instances with large $\tau$ where the tardiness costs tend to dominate the earliness costs.

## 6. Conclusion

This work has addressed several aspects of solving the one-machine scheduling problem with setup and earliness–tardiness penalties. From MIP formulations, several lower bounds can be derived but it is not sufficient to have an efficient algorithm to solve problems with more than 10 tasks. Based on a fast lower bound and good dominance rules, our branch-and-bound approach outperforms any of the MIP approaches. However, the branch-and-bound approach is limited to problems with no more than 20 jobs. In order to solve practical problems, whose size is between 100 and 500 jobs, the proposed multi-start heuristic seems well adapted. In particular, this algorithm found the best known results for "real-life" instances with 24 up to 500 tasks proposed by ILOG researchers [33].

## Acknowledgements

The author would like to thank Jérôme Rogerie and Claude Le Pape from ILOG and an anonymous referee that helped to improve the presentation of the paper.

## References

[1] Webster ST. The complexity of scheduling job families about a common due date. Operations Research Letters 1997;20:65–74.
[2] Chen ZL. Scheduling with batch setup times and earliness–tardiness penalties. European Journal of Operational Research 1997;97:518–37.
[3] Baker KR, Scudder G. Sequencing with earliness and tardiness penalties: a review. Operations Research 1990;38: 22–36.
[4] T'kindt V, Billaut J-C. Multicriteria scheduling: theory, models and algorithms. Berlin: Springer; 2002.
[5] Allahverdi A, Gupta JND, Aldowaisan T. A review of scheduling research involving setup considerations. OMEGA International Journal of Management Science 1999;27:219–39.
[6] Fry TD, Armstrong RD, Blackstone JH. Minimize weighted absolute deviation in single machine scheduling. IIE Transactions 1987;19:445–50.
[7] Garey MR, Tarjan RE, Wilfong GT. One-processor scheduling with symmetric earliness and tardiness penalties. Mathematics of Operations Research 1988;13:330–48.
[8] Davis JS, Kanet JJ. Single machine scheduling with early and tardy completion costs. Naval Research Logistics 1993;40:85–101.
[9] Szwarc W, Mukhopadhyay SK. Optimal timing schedules in earliness–tardiness single machine sequencing. Naval Research Logistics 1995;42:1109–14.
[10] Chrétienne Ph, Sourd F. Scheduling with convex cost functions. Theoretical Computer Science 2003;292:145–64.
[11] Sourd F. Scheduling a sequence of tasks with general completion costs. Research Report 2002/013, LIP6, 2002.
[12] Kim YD, Yano C. Minimizing mean tardiness and earliness in single-machine scheduling problems with unequal due dates. Naval Research Logistics 1994;41:913–33.
[13] Hoogeveen JA, van de Velde SL. A branch-and-bound algorithm for single-machine earliness–tardiness scheduling with idle time. INFORMS Journal on Computing 1996;8:402–12.
[14] Sourd F, Kedad-Sidhoum S. The one machine problem with earliness and tardiness penalties. Journal of Scheduling 2003;6:533–49.
[15] James R, Buchanan J. A neighborhood scheme with a compressed solution space for the early/tardy scheduling problem. European Journal of Operational Research 1997;102:513–27.
[16] Wan G, Yen BPC. Tabu search for single machine scheduling with distinct due windows and weighted earliness/tardiness penalties. European Journal of Operational Research 2002;142:271–81.

[17] Ventura JA, Radhakrishnan S. Single machine scheduling with symmetric earliness and tardiness penalties. European Journal of Operational Research 2003;144:598–612.
[18] Azizoglu M, Webster S. Scheduling job families about an unrestricted common due date on a single machine. International Journal on Production Research 1997;35:1321–30.
[19] Rabadi G, Mollaghasemi M, Anagnostopoulos GC. A branch-and-bound algorithm for the early/tardy machine scheduling problem with a common due-date and sequence-dependent setup time. Computers & Operations Research 2004; in press doi:10.1016/S0305-0548(03)00117-5.
[20] Webster ST, Jog PD, Gupta A. A genetic algorithm for scheduling job families on a single machine with arbitrary earliness/tardiness penalties and an unrestricted common due date. International Journal on Production Research 1998;36:2543–51.
[21] Balakrishnan N, Kanet JJ, Sridharan SV. Early/tardy scheduling with sequence dependent setups on uniform parallel machines. Computers & Operations Research 1999;26:127–41.
[22] Zhu Z, Heady RB. Minimizing the sum of earliness/tardiness in multi-machine scheduling: a mixed integer programming approach. Computers & Industrial Engineering 2000;38:297–305.
[23] ten Kate HA, Wijngarrd J, Zijm WHM. Minimizing weighted total earliness, total tardiness and setup costs. SOM Research Report 95A37, SOM—University of Groningen, 1995.
[24] Yano CA, Kim YD. The optimal operation of mixed production facilities-extensions and improvements. European Journal of Operational Research 1991;52:167–78.
[25] Queyranne M, Schulz AS. Polyhedral approaches to machine scheduling. Technical Report 408-1994, Technische Universität Berlin, 1994.
[26] ILOG Inc., ILOG CPLEX 8.1 user's manual and reference manual. 2002.
[27] van den Akker JM, Hurkens CAJ, Savelsbergh MWP. Time-indexed formulations for machine scheduling problems: column generation. INFORMS Journal on Computing 2000;12:111–24.
[28] Péridy L, Pinson E, Rivreau D. Using short-term memory to minimize the weighted number of late jobs on a single machine. European Journal of Operational Research 2003;148:591–603.
[29] Baptiste Ph, Demassey S. Tight LP bounds for resource constrained project scheduling. OR-Spectrum 2004; to appear.
[30] Sourd F. The continuous assignment problem and its application to preemptive and non-preemptive scheduling with irregular cost functions. INFORMS Journal on Computing, 2004; 16(2).
[31] Lemaréchal C. Nondifferentiable optimization. In: Nemhauser GL, Rinnooy Kan AHG, Todd MJ, editors. Optimization. Handbooks in operations research and management science, vol. 1. Amsterdam: North-Holland; 1989.
[32] ILOG Inc., ILOG Scheduler 5.3 user's manual and reference manual. December 2002.
[33] Nuijten W, Bousonville T, Foccaci F, Godard D, Le Pape C. Towards a real-life manufacturing scheduling problem and test bed. Proceedings of PMS'04, 2004; submitted.

**Francis Sourd** is a researcher at CNRS (the French National Research Foundation) and works for LIP6 (Laboratory of Computer Science of the University Pierre et Marie Curie). He received his PhD in Computer Science from University Pierre and Marie Curie in 2000 and then worked for ILOG Inc. as a full-time scientist then as an independent contractor. His research interests include scheduling problems such as job-shop and earliness–tardiness scheduling and generally applied combinatorial optimization problems.